Supuestos – Proyecto Final Bases de Datos 2: Tienda de Ropa

Oct 26, 2026 · @Juan Mateus

Estos supuestos definen las reglas del negocio que el modelo de datos da por ciertas. Cada uno indica cómo se refleja en el diseño, para que el modelo E-R, el diccionario de datos y la lógica en PL/pgSQL sean coherentes entre sí.

1. Operación del negocio

La tienda es un solo punto de venta físico que vende prendas al detal, atendido por empleados que registran las ventas en caja.

ID	Supuesto	Cómo se refleja en el diseño
S-01	La tienda tiene una sola sede física y una sola bodega.	No existe entidad SEDE ni BODEGA; el stock se guarda directamente en cada variante.
S-02	Toda venta es presencial y la registra un empleado en caja.	VENTA tiene `id_empleado` obligatorio (NOT NULL).
S-03	Los empleados tienen uno de tres cargos: cajero, administrador o analista.	EMPLEADO.cargo con CHECK sobre esos tres valores; cada cargo corresponde a un rol de base de datos.
S-04	Un empleado inactivo no se borra, se desactiva.	EMPLEADO tiene un campo `activo`; las ventas antiguas conservan su referencia.
S-05	La tienda compra mercancía a varios proveedores, y un mismo producto puede venir de más de un proveedor a costos distintos.	Relación N:M PRODUCTO–PROVEEDOR resuelta con PRODUCTO_PROVEEDOR, que guarda el costo.
S-06	Las compras a proveedores no se gestionan como órdenes de compra; solo se registra la entrada de mercancía.	No existe entidad ORDEN_COMPRA; la entrada se registra con `sp_entrada_inventario`.

2. Catálogo e inventario

El inventario se controla por variante (producto + talla + color), y cada variante separa las unidades que se pueden vender de las averiadas.

ID	Supuesto	Cómo se refleja en el diseño
S-07	Cada producto pertenece a una sola categoría (camisas, jeans, chaquetas, etc.).	Relación 1:N CATEGORIA–PRODUCTO; PRODUCTO.id_categoria es NOT NULL.
S-08	Un producto se vende en varias combinaciones de talla y color; cada combinación es una variante con su propio SKU.	VARIANTE_PRODUCTO con UNIQUE (id_producto, id_talla, id_color) y UNIQUE (sku).
S-09	Las tallas son un catálogo fijo: XS, S, M, L, XL, XXL y tallas numéricas de pantalón (28 a 40).	Tabla TALLA con valores precargados; no se escribe la talla como texto libre.
S-10	El precio se define por producto, no por variante: todas las tallas y colores cuestan lo mismo.	PRODUCTO.precio_base; VARIANTE no tiene precio.
S-11	Una prenda averiada (manchada, rota, sin botón o con defecto de fábrica) no se puede vender hasta que se resuelva.	VARIANTE tiene `stock_disponible` y `stock_averiado`; solo el disponible se descuenta en ventas.
S-12	Al registrar una avería se indica la cantidad y el motivo; las unidades pasan de disponible a averiado.	`sp_registrar_averia` dentro de una transacción; el motivo queda en MOVIMIENTO_INVENTARIO.
S-13	Las unidades averiadas no se reparan dentro del sistema; se devuelven al proveedor o se dan de baja.	Movimiento de tipo BAJA_AVERIA que descuenta `stock_averiado`.
S-14	El stock nunca puede ser negativo.	CHECK (stock_disponible >= 0 AND stock_averiado >= 0) y trigger que lanza una excepción antes de violarlo.
S-15	Cada variante tiene un stock mínimo; al bajar de él se genera una alerta de reposición.	VARIANTE.stock_minimo y trigger de alerta.

ID	Supuesto	Cómo se refleja en el diseño
S-16	Todo cambio de stock queda registrado con su tipo: ENTRADA, VENTA, ANULACION, AVERIA o BAJA_AVERIA.	MOVIMIENTO_INVENTARIO lo llena un trigger; tipo con CHECK sobre esos valores.
S-17	Un producto que se deja de vender se desactiva, no se borra, para conservar el historial de ventas.	PRODUCTO.activo; las llaves foráneas usan ON DELETE RESTRICT.

3. Ventas y pagos

Una venta es una operación atómica: o se registra completa (venta, detalles, pagos, inventario y fidelidad), o no se registra nada.

ID	Supuesto	Cómo se refleja en el diseño
S-18	Una venta puede incluir varias variantes, cada una con su cantidad.	Relación N:M VENTA–VARIANTE resuelta con DETALLE_VENTA (cantidad > 0).
S-19	El precio unitario se guarda en el detalle en el momento de la venta, porque el precio del producto puede cambiar después.	DETALLE_VENTA.precio_unitario se copia de PRODUCTO.precio_base al registrar.
S-20	Una venta se puede pagar con uno o varios métodos: efectivo, tarjeta débito, tarjeta crédito o transferencia.	Relación 1:N VENTA–PAGO; PAGO.metodo con CHECK sobre esos valores.
S-21	La suma de los pagos debe ser igual al total de la venta.	`sp_registrar_venta` valida la suma y hace ROLLBACK si no cuadra.
S-22	Los pagos se registran pero no se procesan con una pasarela real.	PAGO guarda método y valor, sin datos de tarjeta.
S-23	Una venta puede hacerse a un cliente registrado o a un cliente ocasional sin registro.	VENTA.id_cliente admite NULL; sin cliente no aplican beneficios de fidelidad.
S-24	Si una variante no tiene stock disponible suficiente, la venta completa se rechaza.	`sp_registrar_venta` bloquea la fila de la variante (SELECT … FOR UPDATE), valida y lanza una excepción con ROLLBACK.

ID	Supuesto	Cómo se refleja en el diseño
S-25	Una venta no se borra; se anula. Al anularla, las unidades vuelven al stock disponible y se revierte lo sumado a la fidelidad del cliente.	VENTA.estado con CHECK ('ACTIVA', 'ANULADA'); `sp_anular_venta`.
S-26	Solo un administrador puede anular ventas.	Permiso EXECUTE sobre `sp_anular_venta` solo para el rol administrador.
S-27	Los cambios y devoluciones parciales de prendas quedan fuera del alcance; solo se anula la venta completa.	No existe entidad DEVOLUCION.

4. Clientes, fidelización y promociones

Los beneficios a clientes se calculan en la base de datos con una sola función, y nunca se acumulan: se aplica el que más favorece al cliente.

ID	Supuesto	Cómo se refleja en el diseño
S-28	Un cliente se identifica por su documento, que es único.	CLIENTE.documento con UNIQUE.
S-29	Al registrar un cliente se pide su fecha de nacimiento, necesaria para el beneficio de cumpleaños.	CLIENTE.fecha_nacimiento NOT NULL, con CHECK de que no sea una fecha futura.
S-30	Cada cliente registrado tiene exactamente una cuenta de fidelidad, creada al registrarlo.	Relación 1:1 CLIENTE–CUENTA_FIDELIDAD; la PK de la cuenta es también FK al cliente. Un trigger la crea en nivel Clásico.
S-31	Existen tres niveles: Clásico (cualquier cliente), Frecuente (desde $1.500.000 en compras de 12 meses) y VIP (desde $4.000.000).	Tabla NIVEL_FIDELIDAD con `compra_minima_anual` y `pct_cumpleanos`.
S-32	El descuento de cumpleaños aplica durante todo el mes de nacimiento: 20% Clásico, 25% Frecuente, 30% VIP.	`fn_calcular_descuento` compara el mes de la venta con el de `fecha_nacimiento`.
S-33	El nivel se recalcula después de cada venta o anulación con lo comprado en los últimos 12 meses; puede subir o bajar.	Trigger `trg_actualizar_nivel` sobre VENTA.

ID	Supuesto	Cómo se refleja en el diseño
S-34	Las promociones de bono entregan un valor fijo cuando el subtotal alcanza un monto mínimo (ej.: $70.000 desde $300.000) y tienen vigencia.	Tabla PROMOCION con `monto_minimo`, `valor_bono`, `fecha_inicio`, `fecha_fin` y `activa`.
S-35	Solo puede haber una promoción de bono activa a la vez.	Índice único parcial sobre PROMOCION donde `activa = true`.
S-36	Los descuentos no son acumulables: si el cliente califica para cumpleaños y para bono, se aplica el de mayor valor.	`fn_calcular_descuento` calcula ambos y devuelve el mayor; VENTA guarda `tipo_descuento` e `id_promocion`.
S-37	El descuento nunca puede superar el subtotal de la venta.	CHECK (descuento <= subtotal) en VENTA.
S-38	Las reseñas de productos solo las escriben clientes registrados que compraron ese producto.	Validación en la aplicación antes de insertar en la colección `resenas` de MongoDB.

5. Datos y formatos

Los datos siguen las convenciones colombianas y se validan en la base de datos, no solo en la aplicación.

ID	Supuesto	Cómo se refleja en el diseño
S-39	Todos los valores monetarios están en pesos colombianos (COP), sin centavos.	Tipo NUMERIC(12,0) en precios, costos, descuentos, pagos y totales.
S-40	Las fechas y horas se registran en la zona horaria de Bogotá (UTC−5).	Columnas TIMESTAMP con zona horaria America/Bogota configurada en el servidor.
S-41	El documento del cliente puede ser cédula de ciudadanía, cédula de extranjería o pasaporte.	CLIENTE.tipo_documento con CHECK ('CC', 'CE', 'PA').
S-42	El NIT del proveedor es único.	PROVEEDOR.nit con UNIQUE.
S-43	Los correos electrónicos tienen un formato válido y no se repiten entre clientes.	CHECK con expresión regular sobre `email`; UNIQUE en CLIENTE.email.

ID	Supuesto	Cómo se refleja en el diseño
S-44	Todas las llaves primarias son números generados automáticamente, excepto la de CUENTA_FIDELIDAD, que es la del cliente.	Columnas GENERATED ALWAYS AS IDENTITY.
S-45	Los datos de prueba son ficticios: no se usan datos personales reales de clientes ni marcas reales.	Script de inserción con nombres, documentos y productos inventados.
S-46	Para los reportes, la base de datos tiene ventas de al menos 12 meses.	El script de inserción genera ventas fechadas entre noviembre de 2025 y noviembre de 2026.

6. Técnicos y de seguridad

La solución corre completa en una máquina virtual local y la seguridad se aplica con roles de base de datos.

ID	Supuesto	Cómo se refleja en el diseño
S-47	PostgreSQL, MongoDB y la aplicación corren en la misma máquina virtual con Ubuntu Server LTS, en VirtualBox.	Una sola VM documentada en la carpeta `/vm` del repositorio.
S-48	Se usan versiones estables recientes de PostgreSQL, MongoDB Community y Node.js LTS.	Las versiones exactas se registran en la guía de instalación.
S-49	La aplicación no ejecuta lógica de negocio: llama a los procedimientos y funciones de PostgreSQL.	La aplicación usa `CALL sp_…` y `SELECT fn_…`; no inserta directamente en VENTA ni en VARIANTE.
S-50	Existen tres roles de base de datos: `cajero` (registra ventas y consulta catálogo), `administrador` (todo, incluido anular ventas y gestionar catálogo) y `analista` (solo lectura y reportes).	CREATE ROLE y GRANT por rol en el script de seguridad.
S-51	Ningún rol de la aplicación tiene permisos de superusuario ni puede modificar la estructura de las tablas.	La aplicación se conecta con usuarios que heredan uno de los tres roles.

ID	Supuesto	Cómo se refleja en el diseño
S-52	Las contraseñas de conexión no se guardan en el repositorio.	Archivo `.env` excluido en `.gitignore`.
S-53	En MongoDB se guardan datos de estructura variable que no requieren transacciones estrictas: reseñas, carritos abandonados y fichas técnicas.	Colecciones `resenas`, `carritos` y `fichas_tecnicas`, enlazadas por SKU o id de producto.
S-54	La relación entre ambas bases se mantiene por identificadores, no por llaves foráneas.	Los documentos de MongoDB guardan `id_producto` o `sku`; la aplicación verifica que existan en PostgreSQL.
S-55	Los respaldos se hacen manualmente con las herramientas nativas de cada motor.	`pg_dump` y `mongodump`, documentados en la guía de la VM.

7. Proyecto y restricciones

El proyecto se desarrolla con un equipo de dos personas en 16 días, lo que limita el alcance a lo exigido por el enunciado.

ID	Supuesto o restricción	Consecuencia
S-56	El equipo lo forman dos integrantes, sujeto a la aprobación del docente.	La carga se divide en dos frentes: diseño y lógica relacional (Juan); gestión, infraestructura y MongoDB (Diego). La aplicación se reparte entre ambos.
S-57	El plazo va del 26 de octubre al 11 de noviembre de 2026, con sustentación el 20 de noviembre.	Las funcionalidades fuera del alcance de la propuesta no se desarrollan.
S-58	Oracle no está permitido como motor relacional.	Se usa PostgreSQL; la lógica vista en clase en PL/SQL se traslada a PL/pgSQL.
S-59	La aplicación es una demostración funcional, no un producto en producción.	Se priorizan las pantallas que evidencian la integración con ambas bases sobre el diseño visual.
S-60	Si el tiempo se acorta, se recortan pantallas de la aplicación antes que lógica en la base de datos.	Procedimientos, triggers y transacciones se completan primero porque concentran la calificación.

ID	Supuesto o restricción	Consecuencia
S-61	Todas las herramientas son gratuitas o de licencia académica.	VirtualBox, Ubuntu Server, PostgreSQL, MongoDB Community, Node.js, MySQL Workbench o pgModeler, GitHub.
S-62	El trabajo se integra mediante Pull Requests a la rama `develop` y cada integrante hace commits desde su propia cuenta.	La participación de ambos queda visible en el historial de GitHub.